

Lead Management System

Mini CRM - Full Stack Assignment Report

GitHub Repository:

Live Application (FE):

Live Backend URL (BE):

Tech Stack:

https://github.com/rajpratham1/Sankar_Group_Assignment

<https://sankar-group-assignment-2i8r.vercel.app/>

<https://sankar-group-assignment.vercel.app/api>

React (Frontend) + Node.js/Express (Backend) + PostgreSQL (DB)

1. Project Overview & Objectives

The objective of this assignment was to build a full-stack Lead Management System (Mini CRM) enabling users to create, view, update, and manage sales leads. The application integrates a modular React frontend with a secure Node.js & Express REST API, communicating with a PostgreSQL relational database. The goal was to write clean, maintainable, and industry-standard code, optimize it for cloud deployment, and solve common runtime and network issues (such as CORS).

2. Features Built vs. Requirements Checklist

Module / Feature	Implemented Functionality	Status
Add Lead Form	Create leads with Name, Phone, Source & Notes.	Completed
Lead List View	Tabular view of leads with creation timestamps.	Completed
Status Updates	Inline status changes (Interested, Converted, etc.)	Completed
REST API CRUD	Express API endpoints for GET, POST, PATCH, DELETE.	Completed
PostgreSQL Integration	Database connection pool, schema seeding & migrations.	Completed
Validation Rules	Duplicate phone checks & fields validation on FE & BE.	Completed
Search / Filters	Bonus: Search by name/phone & filter by status/source.	Completed
CRM KPI Dashboard	Bonus: Cards showing total leads & conversion rate.	Completed
Vercel Deployment	Bonus: Deployed both FE and BE apps to Vercel.	Completed

3. Technical Architecture & Implementation Details

Frontend Layer (React 18 & Axios)

- Clean User Interface built with modern dark UI styling, high-fidelity responsive design, and smooth transitions.
- Custom React hooks encapsulate API communication, loading indicators, and validation structures.
- React Hot Toast integrated to provide micro-animations and status notifications for all user CRUD actions.

Backend API Layer (Node.js & Express)

- Structured API paths namespace prefixed under '/api/leads'.
- Configured Express body parsers and sanitization validation chains using 'express-validator' middleware.
- Integrated error handling catch-alls to safely catch runtime SQL constraints and prevent API crashes.

Database Layer (PostgreSQL)

- Automated Table Seeding: On backend initialization, a check ensures the `leads` table and constraints are populated.
- Development Mocking: Equipped with an optional `pg-mem` in-memory database configuration which allows the system to run locally out-of-the-box without requiring a running PostgreSQL daemon.
- Connection String Support: Enabled `DATABASE\_URL` parsing with SSL connection options, allowing immediate compatibility with serverless Postgres providers (e.g. Neon, Supabase, Vercel Postgres).

4. Critical Problem Solving & Vercel Adjustments

During live deployment, we resolved several production-critical challenges to make the project Vercel-ready:

1. CORS Header Normalization (Trailing Slashes):

The application was failing to load resources due to a CORS mismatch between origin URLs (with and without trailing slashes). We configured a customized CORS middleware in 'src/index.js' that automatically inspects, normalizes, and matches the incoming origin headers against an array of accepted variants, resolving browser blocks completely.

2. Express Serverless Adapter:

Express app listeners (app.listen) block Vercel's serverless compilation environment. We exported the raw 'app' instance as a module and conditionalized 'app.listen()' to trigger only during direct command execution (npm run dev). We also added a 'vercel.json' router file mapping wildcard paths correctly to the serverless endpoint.

3. Cloud Database Integrations:

Added standard SSL configurations to the PG client pool so that the application remains secure and integrates seamlessly with Neon, Supabase, or AWS RDS without SSL verification issues.

5. Conclusion & Final Status

The Lead Management CRM assignment was built to fulfill all the standard parameters of a professional web application. By going beyond the requirements to implement advanced database pooling, custom CORS normalizations, serverless compatibility, dashboard KPIs, and dynamic filters, the system is fully production-ready, hosted, and secure. All code is formatted cleanly, packaged, and pushed to the repository.

Report Prepared by: Assignment Contributor

Role: Full Stack Engineer

Status: Completed and Ready for Review